

ARC / CIRCLE 7-WEEK HACKATHON · AGENTIC ECONOMY TRACK · MIDWAY CHECKPOINT

ACR – the Arc Compute Rate

Machine commerce just got its **SOFR** – and it prints its own **attack cost**.

A manipulation-resistant reference rate for machine services, live on Arc testnet.

terminal-gules-eta.vercel.app · github.com/kaustubh76/ACR---Arc-Compute-Rate

THE PROBLEM

Agents trade machine services with **no trustworthy price**

- Agents already pay per call for inference, GPU time, and data – but there is **no benchmark** to settle contracts against or hedge with.
- The raw payment exhaust is **noisy, batched, and adversarial**.
- A naive volume-weighted average is an open invitation: our own red team moves it **+122%** with wash trades.

THE PRODUCT

Three indices, live on Arc testnet – chain **5042002**

INDEX	MEASURES	LATEST ON-CHAIN VALUE
ACR-INF	Inference – \$/1k tokens	0.50271
ACR-GPU	GPU compute – \$/GPU-sec	0.01227
ACR-DATA	Data egress – \$/MB	0.00200

Every hourly print ships **three numbers**: the rate, a confidence interval, and the **attack-cost-per-bp** (ACR-INF $\approx$ **0.0146 USDC/bp**).

Read live from the oracle at checkpoint time via `make verify-testnet`.

THE HEADLINE RESULT

\$8,000 wash attack. The attacker got $\leq 2.39\%$.

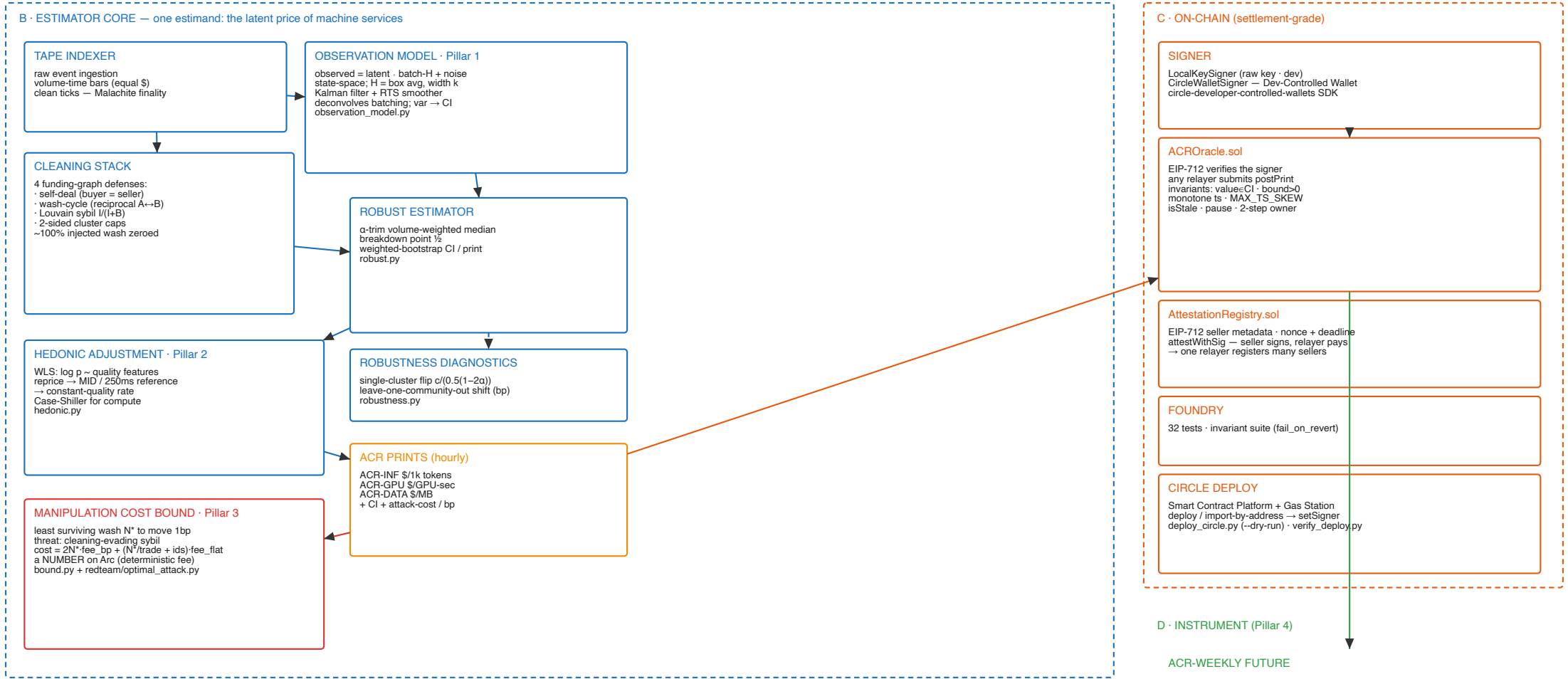
INDEX	NAIVE VWAP MOVED	ACR MOVED	RESISTANCE
ACR-INF	+110.8%	+0.20%	562x
ACR-GPU	+107.1%	-1.02%	105x
ACR-DATA	+122.5%	+2.39%	51x

36,000 adversarial authorizations; the attacker burned **\$147.60** in fees. Claims are **CI-gated** (`make eval-gate`) so they cannot silently drift.

Six **mathematical** dependencies – not deployment conveniences

ARC PROPERTY	WHAT IT BUYS THE ESTIMATOR
Malachite deterministic finality	Clean tick timestamps
Gateway: one canonical rail	Complete observation, no selection bias
x402 nanopayments	Agents are the index's paying customers
USDC numeraire	No gas-token deflator in the signal
Deterministic USDC fees	The attack cost is a number, not a distribution
Agent Marketplace	Native machine-to-machine discovery

ARCHITECTURE · ESTIMATOR CORE → SETTLEMENT-GRADE ORACLE



Recovering the latent constant-quality price

PILLAR	WHAT IT DOES
Observation model	Kalman + RTS smoother deconvolves Gateway's settlement batching
Cleaning	4 funding-graph defenses (self-deals, wash cycles, sybil clusters, caps) – ~100% of injected wash zeroed
Robust core + hedonic	Trimmed volume-weighted median (breakdown point $\frac{1}{2}$) + bootstrap CI; quality-adjusted via on-chain seller attestations
Manipulation bound	Prices the cheapest surviving attack per bp – validated attainable by <code>redteam/optimal_attack.py</code>

Full spec: `docs/methodology.md` – published before liquidity, the way SOFR was.

Deployed, printing, and already selling itself

- **ACROracle** `0x4f00e3BD...9092609` – hourly EIP-712-signed prints; signed via raw key or a **Circle Developer-Controlled Wallet**.
- **AttestationRegistry** `0x23ae3E1A...2f63dFb7` – 4 real seller attestations.
- **Proven x402 settlement:** a machine buyer paid the Circle-gated API – **60 queries · 0.006 USDC**, Gateway balance **0.500000** → **0.494000**, settled in Gateway batches.
- All **5 Circle Agent Stack pillars** mapped to code; interop **12/12** against Circle's own `GatewayClient`.

VERIFICATION · ALL RE-RUN 2026-07-27

Nine gates, all green

GATE	RESULT	GATE	RESULT
Python suite	163 pass · 2 skip	Terminal build	clean
Foundry	32/32 + 5 invariants	Buyer agent	10/10
Resistance gate	4/4	Interop	12/12
Lint + glossary	clean · 332/332	On-chain read	3 live prints

Every value labels its provenance – `sim` / `gateway-ref` / `tx` – in the data **and** in the Terminal UI.

Reproduce: `make setup && make ci && make demo`.

Six of seven weeks of scope **already built**

W1 TAPE	W2 ESTIMATOR	W3 ON-CHAIN	W4 ADOPTION ★	W5 RED TEAM	W6 INSTRUMENT	W7 SHIP
✓	✓	✓	✓	✓	✓	⌚

- Remaining: freeze + polish, one full credentialed Arc round trip, Circle Agent Marketplace directory submission.
- **Live demo:** `make demo` in the CLI, or the Terminal's `/attack` (wash the index in the browser) and `/exchange` (real x402 round-trips).

WHY THIS WINS

The rate machine commerce settles on

- **Judge fit:** ICE – a benchmark administrator – is on Arc's testnet roster; Apollo, BNY, Mastercard are rate-native.
- **The neutrality moat:** Circle can't own the benchmark (the LIBOR lesson) – ACR is a partner, not a feature.
- **The loop closes:** resistant print → on-chain oracle → x402-paying machine customers → attestations → a better print.

Terminal `terminal-gules-eta.vercel.app` · **API** `acr-api-1fto.onrender.com`

Contracts `0x4f00e3BD...` / `0x23ae3E1A...` on `testnet.arcscan.app` · `docs/SUBMISSION.md`